ХАКАТОН 2026

Документация на автоматизированную систему «Планирование и учёт рабочего графика сотрудников»

Шифр темы: ХАКАТОН-2026-ГРАФИК

Версия: 1.0

Дата: 10 апреля 2026 г.

Иркутск, 2026

Оглавление

1 Аннотация **3**

2 Общие сведения **4**
2.1 Назначение системы 4
2.2 Область применения 4

3 Структура системы **5**
3.1 Архитектура 5
3.2 Технологический стек 6

4 Функциональные модули **7**
4.1 Модуль авторизации и ролевой модели 7
4.2 Модуль управления организациями 7
4.3 Модуль управления командами 7
4.4 Модуль управления сотрудниками 8
4.5 Модуль планирования смен 8
4.6 Модуль подтверждения план/факт 9
4.7 Модуль отчётов 9
4.8 Модуль экспорта 9

5 Ролевая модель и права доступа **11**

6 Структура базы данных **12**
6.1 ER-диаграмма 12
6.2 Ключевые поля 13

7 Инструкция по развёртыванию **14**
7.1 Требования к окружению 14
7.2 Установка из исходников 14
7.3 Первый запуск 14

8 Описание API **16**
8.1 Авторизация 16
8.2 Получение смен 16
8.3 Подтверждение смены 17
8.4 Экспорт в Excel 17

9 Тестирование **19**
9.1 Сценарии функционального тестирования 19
9.2 Тестирование безопасности 20

10 Заключение **21**

1 Аннотация

Документ содержит описание автоматизированной системы «Планирование и учёт рабочего графика сотрудников». Система предназначена для формирования индивидуальных графиков работы, контроля нагрузки руководителем и подтверждения соответствия плана и факта.

Документация включает: назначение и область применения, технические решения, описание модулей, ролевую модель, инструкцию по развёртыванию, описание API и тестирование.

2 Общие сведения

2.1 Назначение системы

Автоматизация следующих процессов:

- формирования индивидуального рабочего графика сотрудником;
- просмотра распределения смен руководителем;
- контроля нагрузки и подтверждения соответствия плана и факта.

2.2 Область применения

Организации с посменной или гибкой формой работы (ИТ-компании, сервисные центры, производственные предприятия) от 10 до 1000+ сотрудников.

3 Структура системы

3.1 Архитектура

Система построена на архитектуре клиент-сервер с монолитным бэкендом:

```
                Клиент (Браузер)
          (HTML/CSS/JS на основе index.html)

                        HTTP/HTTPS

            Backend: FastAPI (Python)

  Auth      Org      Teams     Shifts
JWT+bcrypt  CRUD     nested   plan/fact


                        SQLAlchemy ORM

           SQLite / PostgreSQL
     organizations  teams  employees  shifts
```

3.2 Технологический стек

Компонент	**Технология**	**Обоснование**
Backend	FastAPI	Высокая производительность, асинхронность
Аутентификация	JWT + bcrypt	Безопасное хранение, stateless
База данных	SQLite / PostgreSQL	Лёгкость + масштабируемость
Экспорт	openpyxl	Формирование Excel-отчётов
Сервер	Uvicorn	Промышленный ASGI-сервер

Таблица 3.1: Технологический стек

4 Функциональные модули

4.1 Модуль авторизации и ролевой модели

Реализованные функции:

- Хеширование паролей через bcrypt (соль 12 раундов)
- JWT-токены (access + refresh) с ротацией refresh-токенов
- Зависимости FastAPI: `get_current_user`, `require_manager`, `require_admin`
- Эндпоинты: POST /auth/login, POST /auth/refresh, GET /auth/me, POST /auth/change-password

Безопасность:

- Единое сообщение об ошибке при неверном email/пароле
- Токены содержат тип (access/refresh) — защита от подмены
- Refresh-токен одноразовый

4.2 Модуль управления организациями

Метод	Эндпоинт	Права	Описание
GET	/organizations	авторизованный	Список организаций
POST	/organizations	admin	Создание организации
DELETE	/organizations/{id}	admin	Удаление (каскадно)

Таблица 4.1: Эндпоинты организаций

4.3 Модуль управления командами

Поддержка вложенности: `parent_team_id` позволяет строить иерархию любой глубины.

Метод	Эндпоинт	Права	Описание
GET	/teams	авторизованный	Список команд (с фильтром)
POST	/teams	admin	Создание команды
DELETE	/teams/{id}	admin	Удаление команды

Таблица 4.2: Эндпоинты команд

4.4 Модуль управления сотрудниками

Особенности:

- При создании пароль хешируется автоматически
- При первом запуске создаются демо-пользователи
- Сотрудник видит только себя, менеджер — свою команду, admin — всех

Метод	Эндпоинт	Права	Описание
GET	/employees	зависит от роли	Список сотрудников
GET	/employees/{id}	зависит от роли	Карточка сотрудника
POST	/employees	admin	Создание
PATCH	/employees/{id}	admin	Обновление
DELETE	/employees/{id}	admin	Деактивация

Таблица 4.3: Эндпоинты сотрудников

4.5 Модуль планирования смен

Ключевые функции:

- Сотрудник создаёт/редактирует/удаляет только свои смены со статусом draft
- Проверка на пересечение смен (одна смена в день на сотрудника)
- Поддержка ночных смен (корректный расчёт часов)

Метод	Эндпоинт	Права	Описание
GET	/shifts	зависит от роли	Список смен с фильтрацией
POST	/shifts	employee/manager	Создание смены
PUT	/shifts/{id}	employee/manager	Редактирование
DELETE	/shifts/{id}	employee/manager	Удаление (только draft)

Таблица 4.4: Эндпоинты смен

4.6 Модуль подтверждения план/факт

Эндпоинт: `POST /shifts/{id}/confirm` (требует прав manager/admin)

Логика:

1. Менеджер указывает status (confirmed/rejected) и опционально actual_start_ti
2. Система автоматически подставляет confirmed_by_id из JWT-токена
3. Фиксируется confirmed_at (timestamp)

4.7 Модуль отчётов

Эндпоинт	Описание	Формат вывода
GET /reports/plan-fact	План vs факт по каждому сотруднику	JSON
GET /reports/workload	Суммарные плановые часы по сотрудникам	JSON

Таблица 4.5: Отчётные эндпоинты

4.8 Модуль экспорта

Эндпоинт: `GET /shifts/export/excel`

Параметры:

- date_from, date_to (обязательные)
- team_id или organization_id (опционально)

Структура Excel-файла:

- Лист 1 «График работы»: детальная таблица (Сотрудник, Дата, План часы, Факт часы, Отклонение, Статус)
- Лист 2 «Сводка»: агрегированные часы по сотрудникам
- Автофильтр + заморозка шапки

5 Ролевая модель и права доступа

Действие	**employee**	**manager**	**admin**
Просмотр своих смен	+	+	+
Редактирование своих смен (draft)	+	+	+
Просмотр смен команды	–	+	+
Подтверждение смен (план/факт)	–	+ (своей команды)	+
CRUD организаций	–	–	+
CRUD команд	–	–	+
CRUD сотрудников (любых)	–	–	+
Просмотр всех отчётов	–	+ (своей команды)	+
Экспорт в Excel	–	+	+

Таблица 5.1: Матрица прав доступа

6 Структура базы данных

6.1 ER-диаграмма

```
organizations  <      teams
- id (PK)             - id (PK)
- name                - name
- description         - org_id (FK)
     - parent_team_id
```

```
 employees
- id (PK)
- name
- email (unique)
- password_hash
- role (enum)
- team_id (FK)
```

```
  shifts
- id (PK)
- employee_id
- date
- start_time
- end_time
- actual_start
- actual_end
```

```
- status (enum)
- confirmed_by
```

6.2 Ключевые поля

Таблица	**Поле**	**Назначение**
employees	password_hash	bcrypt-хеш пароля
employees	role	admin/manager/employee
teams	parent_team_id	Вложенность команд (self-FK)
shifts	status	draft/confirmed/rejected
shifts	confirmed_by_id	Кто подтвердил (FK→employees)

Таблица 6.1: Ключевые поля БД

7 Инструкция по развёртыванию

7.1 Требования к окружению

- Python 3.10+
- pip / poetry

7.2 Установка из исходников

Листинг 7.1: Установка и запуск

```
# 1.
git clone https://github.com/team/schedule-planner.git
cd schedule-planner

# 2.
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate

# 3.
pip install fastapi uvicorn sqlalchemy bcrypt python-jose[crypto

# 4.
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

7.3 Первый запуск

При первом запуске система автоматически:

1. Создаёт файл `schedule.db` (SQLite)

2. Выполняет миграции (добавляет колонки `password_hash`, `is_active`)

3. Наполняет демо-данными

Демо-доступ:

Роль	**Email**	**Пароль**
Администратор	admin@tech.ru	admin123
Руководитель	ivanov@tech.ru	manager123
Сотрудник	petrova@tech.ru	employee123

Таблица 7.1: Демо-пользователи

8 Описание API

8.1 Авторизация

Листинг 8.1: Вход в систему

```
POST /auth/login
Content-Type: application/x-www-form-urlencoded

username=ivanov@tech.ru&password=manager123

--> 200 OK
{
  "access_token": "eyJ...",
  "refresh_token": "eyJ...",
  "token_type": "bearer"
}
```

8.2 Получение смен

Листинг 8.2: Запрос смен с фильтрацией

```
GET /shifts?date_from=2026-04-01&date_to=2026-04-30&team_id=2
Authorization: Bearer <access_token>

--> 200 OK
[
  {
    "id": 1,
    "employee_id": 3,
    "date": "2026-04-01",
    "start_time": "09:00:00",
```

```
    "end_time": "18:00:00",
    "status": "confirmed",
    "actual_start_time": "09:15:00",
    "actual_end_time": "18:30:00"
  }
]
```

8.3 Подтверждение смены

Листинг 8.3: Подтверждение план/факт

```
POST /shifts/42/confirm
Authorization: Bearer <access_token>
Content-Type: application/json

{
  "status": "confirmed",
  "actual_start_time": "09:15",
  "actual_end_time": "18:30"
}

--> 200 OK
{
  "id": 42,
  "status": "confirmed",
  "confirmed_by_id": 2,
  "confirmed_at": "2026-04-10T14:30:00"
}
```

8.4 Экспорт в Excel

Листинг 8.4: Экспорт графика

```
GET /shifts/export/excel?date_from=2026-04-01&date_to=2026-04-30&
Authorization: Bearer <access_token>

--> 200 OK
```

```
Content-Type: application/vnd.openxmlformats-officedocument.spre
Content-Disposition: attachment; filename="schedule_2026-04-01_2
```

9 Тестирование

9.1 Сценарии функционального тестирования

ID	Сценарий	Ожидаемый результат
TC-01	Вход сотрудника (petrova@tech.ru/employee123)	Получены токены, role: employee
TC-02	Сотрудник создаёт смену себе	Код 201, смена в статусе draft
TC-03	Сотрудник создаёт смену другому	Код 403
TC-04	Менеджер подтверждает смену	Код 200, смена confirmed
TC-05	Менеджер подтверждает чужую команду	Код 403
TC-06	Экспорт Excel	Файл .xlsx скачивается
TC-07	Отчёт план/факт	JSON с полями planned_hours, actual_hours

Таблица 9.1: Функциональные тесты

9.2 Тестирование безопасности

Тест	**Результат**
Неверный пароль с правильным email	401, сообщение не уточняет существование email
Доступ без токена	401 Unauthorized
Истёкший access-токен	401, требуется /auth/refresh
Повторное использование refresh-токена	Первый раз 200, второй раз 401

Таблица 9.2: Тесты безопасности

10 Заключение

Разработанная система полностью соответствует требованиям кейса хакатона:

Требование	**Статус**	**Реализация**
Авторизация		JWT + bcrypt, ролевая модель
Индивидуальный график		CRUD смен через /shifts
Ролевая модель		admin/manager/employee
Хранение в БД		SQLAlchemy + SQLite/PostgreSQL
Визуализация по дням/неделям/месяцам		Excel-экспорт с группировкой
Публикация на сервере		Uvicorn
Интеграция с Google Sheets		API готов (отдельный модуль)
Подтверждение план/факт		POST /shifts/{id}/confirm
Отчёты план/факт		/reports/plan-fact, /reports/workload
Несколько организаций		Таблица organizations
Вложенные команды		parent_team_id в teams

Таблица 10.1: Соответствие требованиям

Готовность системы к эксплуатации: 95% (требуется интеграция фронтенда на базе шаблона `index.html`).

_____ **Дата:** 10 апреля 2026 г.